Computability - 2020 Moed B Solution

Question 1

We show that there does not exist a DFA $\mathcal{A} = \langle \{0,1\}, Q, \delta, q_0, F \rangle$ with $L(\mathcal{A}) = L$, $|F| = 1$ and $|Q| = 4$ such that all states in Q are reachable.

Let $\mathcal{A}$ be a DFA with $L(\mathcal{A}) = L$ and $|F| = 1$. First, since $\varepsilon \in L$, we know that $q_0 \in F$. Let $q_1 = \delta(q_0, 0)$ and $q_2 = \delta(q_0, 1)$ (it is possible that $q_1 = q_2$). Since the words 00, 01, 10, and 11 are all in L, we know that the runs of $\mathcal{A}$ on these words all reach q_0, the single accepting state. It follows that $\delta(q_1, 0) = \delta(q_1, 1) = \delta(q_2, 0) = \delta(q_2, 1) = q_0$. Thus, we get that the transitions from q_0, q_1 and q_2 cannot reach any other state, and there are at most 3 reachable states in Q.

Question 2

The claim is **true**.

Let $L \in$ REG. We have seen in class that REG is closed under union, intersection, and concatenation. Thus, since the language of all even length words is regular, it holds that $L \cap \{w : |w| \text{ is even}\}$ is regular as well. In addition, the language $\{a\}$ is also in REG, and so $\{a\} \cdot (L \cap \{w : |w| \text{ is even}\}) \in$ REG. We note that this language is exactly $\{a \cdot w : w \in L \text{ and } |w| \text{ is even}\}$. With similar considerations, we get that $\{b \cdot w : w \in L \text{ and } |w| \text{ is odd}\}$ is also in REG. Finally, since REG is closed under union, we get that their union, which is exactly $MARK(L)$, is in REG.

Question 3

The claim is **true**.

Let L be a language such that $MARK(L) \in$ REG, and let $\mathcal{A} = \langle \{a,b\}, Q, \delta, q_0, F \rangle$ be a DFA such that $L(\mathcal{A}) = MARK(L)$. Consider the NFA given by $\mathcal{A}' = \langle \{a,b\}, Q, \delta, Q_0 = \{q_a, q_b\}, F \rangle$, where $q_a = \delta(q_0, a)$ and $q_b = \delta(q_0, b)$. Namely, the NFA is obtained by replacing the initial state q_0 with two initial states, q_a and q_b, which are the states reached from q_0 by reading a and b, respectively. We claim that $L(\mathcal{A}') = L$, and conclude that $L \in$ REG.

Let $w \in L$ and $n = |w|$. If n is even then $a \cdot w \in MARK(L)$. Let $q_0, q_1, \ldots, q_n, q_{n+1}$ be the run of $\mathcal{A}$ on $a \cdot w$. It follows that $q_{n+1} \in F$ and $q_1 = q_a$. Consider the run $q_1 = q_a, \ldots, q_n, q_{n+1}$. It is a legal run of A' on w, since $q_a \in Q_0$ and the transition function δ of $\mathcal{A}'$ is the same as that of $\mathcal{A}$. In addition, $q_{n+1} \in F$ and so the run is accepting. Thus, $w \in L(\mathcal{A}')$. If n is odd we use similar reasoning to show that $w \in L(\mathcal{A}')$.

Conversely, let $w \in L(\mathcal{A}')$. Hence, there is an accepting run of $\mathcal{A}'$ on w, starting in either q_a or q_b. Assume w.l.o.g that the run starts in q_a, and consider the run of $\mathcal{A}$ on $a \cdot w$. We have that $\delta^*(q_0, a \cdot w) = \delta^*(\delta(q_0, a), w) = \delta^*(q_a, w)$. Since the run of $\mathcal{A}'$ on w starting in q_a is an accepting run, we have that $\delta^*(q_a, w) \in F$, and so $\delta^*(q_0, a \cdot w) \in F$ and $a \cdot w \in MARK(L)$. By the definition of $MARK(L)$, we have that $w \in L$. If the run of $\mathcal{A}'$ on w starts in q_b, we consider the run of $\mathcal{A}$ on $b \cdot w$, and obtain the desired result using similar considerations.

Question 4

$L_1 \in \text{RE} \setminus \text{R}$. We show that (a) $L_1 \in \text{RE}$ and (b) $L_1 \notin \text{R}$.
First, we describe a TM T that recognizes L_1:

1. On input $\langle M \rangle$, check whether it is a valid encoding of a TM. If not, reject.
2. If it is, simulate M in parallel on all the palindromes. Namely, for $i = 1, 2, \ldots$, simulate M for at most i steps on each of the first i palindromes, in lexicographic order.
3. If M accepts one word, accept.

It is easy to see that T accepts iff M is a TM that accepts at least one palindrome. Thus, $L_1 = L(T) \in \text{RE}$.

We show that $L_1 \notin \text{coRE}$ (and therefore not in R), by showing that $A_{TM} \leq_m L_1$. Given an input $\langle M, w \rangle$, the reduction outputs an encoding of a TM $\langle M' \rangle$, such that M' works as follows:

Given x, run M on w. If M rejects, M' rejects as well. If M accepts, M' accepts only if $x = aba$ (easy to check) and otherwise rejects.

This is a computable reduction, as M''s first step is simulation, and its second step is easily implemented. As for correctness, we need to show that $\langle M, w \rangle \in A_{TM}$ iff $\langle M' \rangle \in L_1$.

1. If $\langle M, w \rangle \in A_{TM}$, then M accepts w and M' accept the word aba so M' accept at least one palindrome. Hence, $\langle M' \rangle \in L_1$.
2. If $\langle M, w \rangle \notin A_{TM}$, M' never accepts, since it can only accept if its simulation of M on w accepts. Thus, $L(M')$ is the empty language. In particular, it does not include any palindrome, and therefore $\langle M' \rangle \notin L_1$.

Hence, $L_1 \notin \text{coRE}$, as required.

Alternative solution: In Questions 4 and 5, we can use Rice's theorem to show that the language is undecidable. In both cases, it is easy to show that these are nontrivial semantic properties.

Question 5

$L_2 \in \text{coRE} \setminus \text{R}$. We show that (a) $L_2 \in \text{coRE}$ and (b) $L_2 \notin \text{R}$.
First, we describe a TM T that recognizes $\overline{L_2}$.

1. On input $\langle M \rangle$, check whether it is a valid encoding of a TM. If not, reject.
2. If it is, simulate M in parallel on all words that are not palindrome. Namely, for $i = 1, 2, \ldots$, simulate M for at most i steps on each of the first i words that are not palindromes, in lexicographic order.
3. If M accepts one word, accept.

It is easy to see that T accepts iff M is a TM that accepts at least one word that is not a palindrome, so $\overline{L_2} = L(T) \in \text{RE}$ and $L_2 \in \text{coRE}$.

We show that $L_2 \notin \text{RE}$ (and therefore not in R), by showing that $\overline{A_{TM}} \leq_m L_2$. Given an input $\langle M, w \rangle$ to $\overline{A_{TM}}$, the reduction outputs an encoding of a TM $\langle M' \rangle$, such that the TM M' works as follows:

Given x, run M on w. If M rejects M' rejects as well. If M accepts, M' accepts only if $x = ab$ (easy to check), and otherwise rejects.

This is a computable reduction, as M''s first step is simulation, and its second step is easily implemented. As for correctness, we need to show that $\langle M, w \rangle \in \overline{A_{TM}}$ iff $\langle M' \rangle \in L_2$.

1. If $\langle M, w\rangle \in \overline{A_{TM}}$, M' never accepts, since it can only accept if its simulation of M on w accept. Thus, $L(M')$ is the empty language. In particular, M' does not accept any w which is not a palindrome, and therefore $\langle M'\rangle \in L_2$.

2. If $\langle M, w\rangle \notin \overline{A_{TM}}$, then M accepts w, and M' accepts the word ba that it is not a palindrome. Thus, $\langle M'\rangle \notin L_2$.

Hence, $L_2 \notin$ RE, as required.

Question 6

The language is in R.
Given a TM M, we accept iff $q_0 \in \{q_{acc}, q_{rej}\}$. Clearly, the check can be done in linear time.

Correctness:

1. If $q_0 \in \{q_{acc}, q_{req}\}$, then M halts immediately on all words, particularly, within at most $|w|$ steps.

2. If M halts on every word w within $|w|$ steps, then in particular it halts on ε within 0 steps, and so $q_0 \in \{q_{acc}, q_{req}\}$.

Remark: We are aware of the fact that the language is similar to an undecidable language you saw in the exercise. Solutions in that direction will be graded somewhat mercifully.

Question 7

The claim is **false**.

Proof: We have seen that PATH is in NL, and NL $\subseteq$ P $\subseteq$ coNP. Hence, PATH is in coNP.

Question 8

The claim is **unknown**.
If it is true, the following claims are also true: P = NP, NP =coNP and P =PSPACE.

Proof: Since there is a polynomial reduction from ALL_{NFA} to PATH, and PATH is decidable in polynomial time (since it is in NL, and in particular, in P), we get that ALL_{NFA} is decidable in polynomial time

We have seen that ALL_{NFA} is PSPACE-hard. Thus, for every L in PSPACE we have $L \leq_p \text{ALL}_{NFA}$. If ALL_{NFA} is decidable in P, we can, by the reduction theorem, decide in polynomial time every language that is in PSPACE. In addition, since both NP and coNP are subsets of PSPACE and supersets of P, it follows that P = NP = coNP.

Note: PATH is in NL, but the reduction in the claim is polynomial time, and not necessarily log-space. Therefore, we cannot conclude anything about NL. A log-space reduction would imply NL = PSPACE, which is false.

Question 9

The claim is **true**.

Proof: We have seen a polynomial reduction from CLIQUE to IS. The reduction "flips" the edges in the graph. That is, for an input $\langle G, k\rangle$, the reduction outputs $\langle \bar{G}, k\rangle$, where the vertices in $\bar{G}$ are the same as in G, and the edge $\{u, v\}$ is an edge in $\bar{G}$ iff $\{u, v\}$ is not an edge in G.

The same reduction is also a reduction from IS to CLIQUE. The correctness immediately follows from the fact that $\bar{\bar{G}} = G$. We claim that it can be computed in logarithmic space. We start by defining some order on the vertices in G. The reduction iterates over $i, j \in \{1, 2, \ldots, |V|\}$ for $i > j$, and checks if there is an edge between the i'th vertex and the j'th vertex in G. If there is not an edge, the TM adds it to the output graph G'. The rest of the output can be copied from the input. Thus,

the only memory space required is for maintaining the counters for i and j, and their maximal value, $|V|$. Since there is at most a polynomial number of vertices, this requires logarithmic space. Thus, we have a log-space reduction from IS to CLIQUE.

Question 10

The language L is NP-complete. We first show $L \in$ NP, then that L is NP-hard.

- The language is in NP since there is a poly-time verifier: The verifier takes a CNF formula φ over $x_1, \ldots, x_n$, two assignments to $x_1, \ldots, x_n$, and $1 \leq i \leq n$. It verifies that:
  - φ is a CNF formula
  - One assignment has $x_i = 0$ and the other $x_i = 1$.
  - Both assignments yield a true value.

  All of these checks can be done in ploy-time (we have seen that evaluating the value of a formula given an assignment takes polynomial time).
- The language is NP-hard since we can show a reduction from CNF-SAT. We have seen that CNF-SAT is NP-hard and from the reduction theorem it follows that L is also NP-hard.

  The reduction takes a CNF formula φ over $x_1, \ldots, x_n$, and outputs $\varphi' = \varphi \wedge (x_{n+1} \vee \neg x_{n+1})$. This can clearly be done in polynomial time. We claim that $\langle \varphi \rangle \in$ CNF-SAT iff $\langle \varphi' \rangle \in L$.
  - If $\langle \varphi \rangle$ is in CNF-SAT then there is an assignment to $x_1, \ldots, x_n$ that satisfies φ. Clearly φ' is satisfied by the same assignment and $x_{n+1} = 0$ or $x_{n+1} = 1$. Thus, for $i = n + 1$, we get that $\langle \varphi' \rangle \in L$.
  - If $\langle \varphi' \rangle$ is in L, then in particular, there is some assignment that satisfies φ'. Since x_{n+1} only appears in the last clause of φ', we get that the same assignment, when ignoring x_{n+1}, satisfies φ. In particular, φ is satisfiable and $\langle \varphi \rangle \in$ CNF-SAT.